

```

# =====
# ✓ DeepLabv3+ Semantic Segmentation (PyTorch)
# =====

import torch
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from google.colab import files
import torchvision.transforms as T
import torchvision

# Check device
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)

# =====
# ✓ Load Pretrained DeepLabv3+ Model
# =====

model = torchvision.models.segmentation.deeplabv3_resnet101(
    weights=torchvision.models.segmentation.DeepLabV3_ResNet101_Weights.DE
FAULT
)
model = model.to(device)
model.eval()

print("DeepLabv3+ Model Loaded!")

# =====
# ✓ Preprocessing Transform
# =====

# Standard ImageNet preprocessing
preprocess = T.Compose([
    T.Resize(520),
    T.ToTensor(),
    T.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

# =====
# ✓ Decode Segmentation Map (21 Pascal VOC Classes)

```

```

# =====
VOC_COLORMAP = np.array([
    [0, 0, 0],      # background
    [128, 0, 0],    # aeroplane
    [0, 128, 0],    # bicycle
    [128, 128, 0],  # bird
    [0, 0, 128],    # boat
    [128, 0, 128],  # bottle
    [0, 128, 128],  # bus
    [128, 128, 128], # car
    [64, 0, 0],     # cat
    [192, 0, 0],    # chair
    [64, 128, 0],   # cow
    [192, 128, 0],  # dining table
    [64, 0, 128],   # dog
    [192, 0, 128],  # horse
    [64, 128, 128], # motorbike
    [192, 128, 128], # person
    [0, 64, 0],     # potted plant
    [128, 64, 0],   # sheep
    [0, 192, 0],    # sofa
    [128, 192, 0],  # train
    [0, 64, 128]    # tv/monitor
])

def decode_segmap(mask):
    h, w = mask.shape
    color_mask = np.zeros((h, w, 3), dtype=np.uint8)

    for label in range(21): # Pascal VOC has 21 classes
        color_mask[mask == label] = VOC_COLORMAP[label]

    return color_mask

# =====
# ✓ Upload Image
# =====

print("Upload an image for segmentation:")
uploaded = files.upload()

for name in uploaded.keys():
    img = Image.open(name).convert("RGB")

```

```

# Preprocess
input_tensor = preprocess(img).unsqueeze(0).to(device)

# Forward pass
with torch.no_grad():
    output = model(input_tensor)["out"][0]

# Convert to mask
mask = output.argmax(0).cpu().numpy()
color_mask = decode_segmap(mask)

# =====
# ✓ Visualization
# =====

plt.figure(figsize=(18, 10))

plt.subplot(1, 3, 1)
plt.title("Original")
plt.imshow(img)
plt.axis("off")

plt.subplot(1, 3, 2)
plt.title("Segmentation Mask")
plt.imshow(color_mask)
plt.axis("off")

plt.subplot(1, 3, 3)
plt.title("Overlay")
plt.imshow(img)
plt.imshow(color_mask, alpha=0.5)
plt.axis("off")

plt.show()

```

Original



Segmentation Mask



Overlay

